

Programmation Web Frontend

Session 1 - Version 2.1

L2/X64/06/PWF/187628/26-27

Votre formateur



Said Eloudrhiri

- Bachelier en électronique
- Bachelier en informatique, spécialisation
- Début de carrière en 1992
- Développeur, Scrum Master, coach agile, Tribe Lead
- Formateur, speaker

 said.eloudrhiri@formateur.ifapme.be



linkedin.com/in/eloudsa

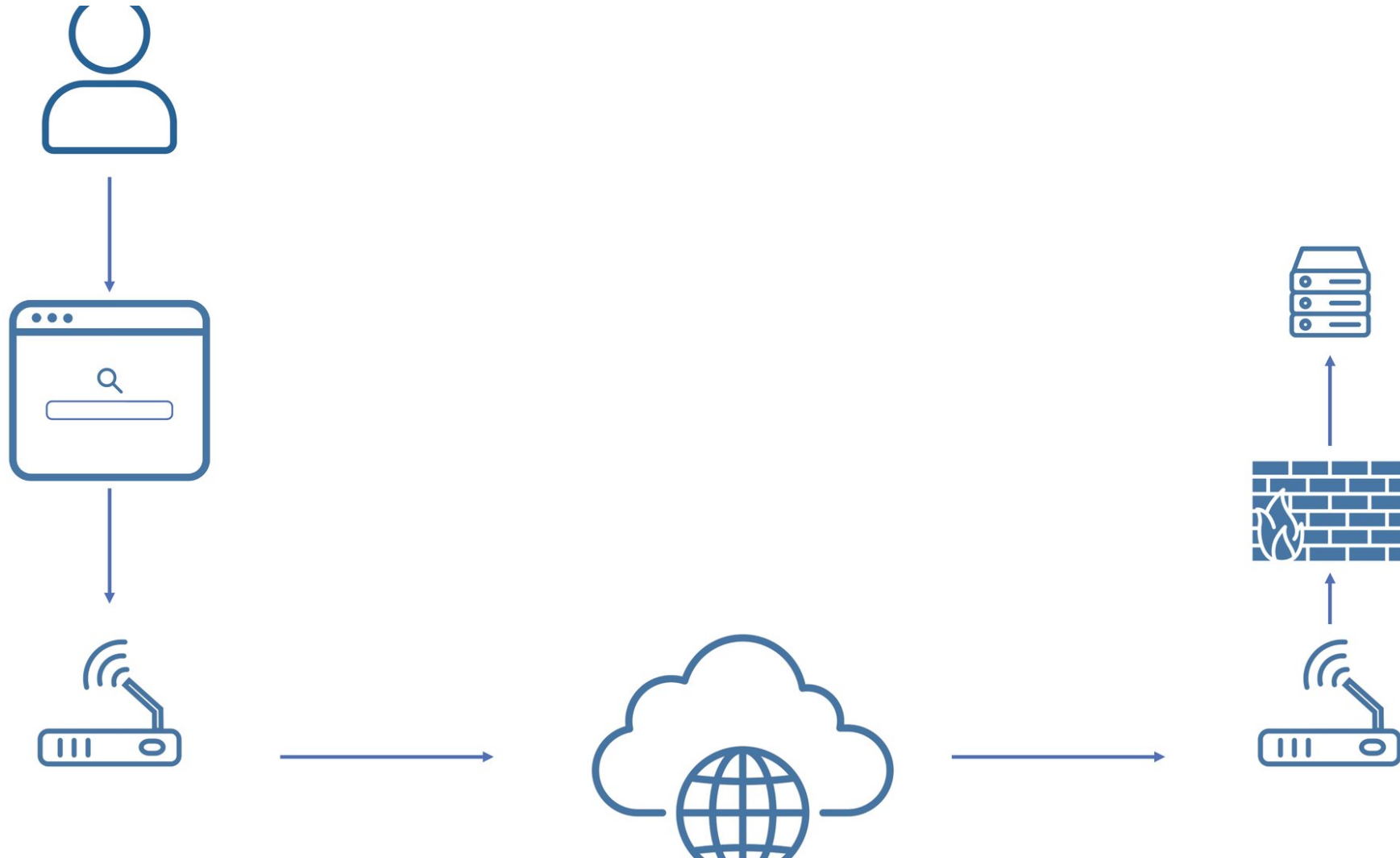
Apprenants



- Vos formations
- Votre activité actuelle
- Vos attentes pour ce cours
- Votre niveau en développement
- Pourquoi UX/UI designer ?

Client et serveur

Requête



Réponse : contenu statique



Réponse : contenu dynamique



Exemple : le suivi des cours dans MyIFAPME



JavaScript

l'interactivité des pages

Programmation Web Frontend

IFAPME Liège — mardi soir

0 séances suivies sur 10

[Voir le programme](#) [Séance suivie](#)

Barre de progression



JavaScript

l'interactivité des pages

Programmation Web Frontend

IFAPME Liège — mardi soir

- delectus aut autem
- quis ut nam facilis et officia qui
- fugiat veniam minus
- et porro tempora
- laboriosam mollitia et enim quasi adipisci quia provident illum
- qui ullam ratione quibusdam voluptatem quia omnis
- illo expedita consequatur quia in
- quo adipisci enim quam ut ab
- molestiae perspiciatis ipsa
- illo est ratione doloremque quia maiores aut

0 séances suivies sur 10

[Masquer](#) [Séance suivie](#)

Action « Voir le programme »



JavaScript

l'interactivité des pages

Programmation Web Frontend

IFAPME Liège — mardi soir

- delectus aut autem
- quis ut nam facilis et officia qui
- fugiat veniam minus
- et porro tempora
- laboriosam mollitia et enim quasi adipisci quia provident illum
- qui ullam ratione quibusdam voluptatem quia omnis
- illo expedita consequatur quia in
- quo adipisci enim quam ut ab
- molestiae perspiciatis ipsa
- illo est ratione doloremque quia maiores aut

4 séances suivies sur 10

[Masquer](#) [Séance suivie](#)

Évolution de la jauge



JavaScript

l'interactivité des pages

Programmation Web Frontend

IFAPME Liège — mardi soir

- delectus aut autem
- quis ut nam facilis et officia qui
- fugiat veniam minus
- et porro tempora
- laboriosam mollitia et enim quasi adipisci quia provident illum
- qui ullam ratione quibusdam voluptatem quia omnis
- illo expedita consequatur quia in
- quo adipisci enim quam ut ab
- molestiae perspiciatis ipsa
- illo est ratione doloremque quia maiores aut

10 séances suivies sur 10

[Masquer](#) [Formation terminée](#)

Changement d'état

Statique ou dynamique ?



Programmation Web Frontend

IFAPME Liège — mardi soir

- delectus aut autem
- quis ut nam facilis et officia qui
- fugiat veniam mi
- et porro tempora
- laboriosam mollitia et enim quasi adipisci quia provident illum
- qui ullam ratione quibusdam voluptatem quia omnis
- illo expedita consequatur quia in
- quo adipisci enim quam ut ab
- molestiae perspiciatis ipsa
- illo est ratione doloremque quia maiores aut



10 séances suivies sur 10

Masquer

Formation
terminée

- **Statique** : la structure de la page, les titres, les couleurs
- **Dynamique** : la jauge qui progresse, le bouton qui change d'état, la recherche dans le programme
- Comment ces deux comportements se combinent-ils dans une même page ?

Sujet de cette formation

HTML (1re année)

```

1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <meta charset="UTF-8">
5   <title>Suivi de formation</title>
6   <link rel="stylesheet" href="style.css">
7 </head>
8 <body>
9
10 <article class="card">
11   <svg class="banner" viewBox="0 0 340 150" role="img" aria-label="JavaScript">
12     <rect width="340" height="150" fill="#16274a"/>
13     <text x="24" y="98" font-size="72" fill="#f8c419" font-family="monospace">{ }</text>
14     <text x="168" y="88" font-size="32" font-weight="bold" fill="white" font-family="sans-serif">JavaScript</text>
15     <text x="288" y="128" fill="white" font-size="12">1 page sur 2 pages</text>
16   </svg>
17   <div class="progress">
18     <h2>Programme</h2>
19     <p>IFAPRE</p>
20     <ul id="program" class="hidden"></ul>
21     <div class="progress-bar"><div id="progress-bar"></div></div>
22     <p id="status">Chargement des séances.</p>
23     <div class="actions">
24       <button id="toggle-program">Voir le programme</button>
25       <button id="attend">Séance suivie</button>
26     </div>
27   </div>
28 </article>
29
30 <script src="script.js"></script>
31
32 </body>
33 </html>

```

CSS (1re année)

```
body { font-family: sans-serif; background-color: #f0f0f0; margin: 0; min-height: 100px; display: flex; align-items: center; justify-content: center; padding: 10px; box-sizing: border-box; }
card { width: 500px; background: white; border-radius: 12px; box-shadow: 0 4px 16px #ccc; margin: 10px; overflow: hidden; }
header { width: 100%; display: flex; justify-content: space-between; padding: 10px; }
content { padding: 10px; }
h2 { margin: 0 0 20px 0; color: #333; font-size: 24px; }
p { margin: 0 0 20px 0; color: #666; font-size: 16px; }
ul { margin: 0 0 20px 0; padding-left: 20px; font-size: 16px; }
li { color: #000; margin-bottom: 10px; }
p:last-child { margin-bottom: 0; }
progress { height: 10px; background-color: #ccc; margin-bottom: 10px; }
progress div { height: 10px; background-color: #007bff; width: 70%; }
button { display: flex; align-items: center; justify-content: center; padding: 10px 20px; border: 1px solid #ccc; border-radius: 6px; }
button:hover { background-color: #f0f0f0; }
button:disabled { background-color: #eee; cursor: default; }
error { color: #dc3545; font-size: 14px; margin-top: 10px; }
div { display: flex; align-items: center; justify-content: center; padding: 10px 0 0 0; }
div span { flex: 1; }
div span span { font-size: 24px; color: #007bff; margin-left: 10px; }
```

JavaScript

```

1 const API:string = "https://jsonplaceholder.typicode.com/todos?userId=1&limit=10";
2 const program:HTMLInputElement = document.getElementById( elementId: "program");
3 const progressBar:HTMLInputElement = document.getElementById( elementId: "progress-bar");
4 const status:HTMLInputElement = document.getElementById( elementId: "status");
5 const toggleProgram:HTMLInputElement = document.getElementById( elementId: "toggle-program");
6 const attend:HTMLInputElement = document.getElementById( elementId: "attend");
7 let sessions:any[] = [];
8
9 Show usages
10 function render():void {
11     const done:number = sessions.filter(s => s.completed).length;
12     program.innerHTML = sessions
13         .map(s => `<li class="${s.completed ? "done" : ""}">${s.title}</li>`)
14         .join("");
15     progressBar.style.width = (done / sessions.length) * 100 + "%";
16     status.textContent = `${done} séances suivies sur ${sessions.length}`;
17     attend.disabled = done === sessions.length;
18     if (attend.disabled) attend.textContent = "Formation terminée";
19 }
20
21 Show usages
22 async function load():Promise<void> {
23     try {
24         const
25         if (
26         sess
27         rend
28     } catch
29     status.textContent = "Impossible de joindre le serveur.";
30     status.classList.add("error");
31 }
32
33 toggleProgram.addEventListener( type: "click", listener: () => void => {
34     program.classList.toggle( token: "hidden");
35     toggleProgram.textContent = program.classList.contains("hidden") ? "Voir le programme" : "Masquer";
36 });
37
38 attend.addEventListener( type: "click", listener: async () => Promise<void> => {
39     const next = sessions.find(s => !s.completed);
40     if (!next) return;
41     await fetch( inputs: "https://jsonplaceholder.typicode.com/todos/${next.id}", init: {
42         method: "PATCH",
43         headers: { "Content-Type": "application/json" },
44         body: JSON.stringify( value: { completed: true } )
45     });
46     next.completed = true;
47     render();
48 });
49
50 load();

```


JavaScript
l'interactivité des pages

Programmation Web Frontend

IFAPME Liège — mardi soir

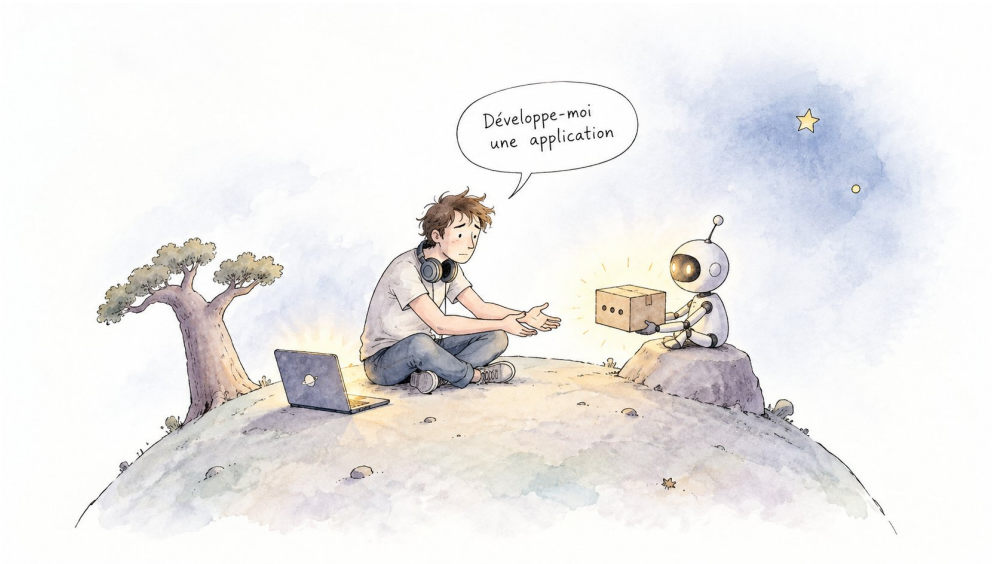
- delectus aut autem
- quis ut nam facilis et officia qui
- fugiat veniam minus
- et porro tempora
- laboriosam mollitia et enim quasi adipisci quia provident illum
- qui ullam ratione quibusdam voluptatem quia omnis
- illo expedita consequatur quia in
- quo adipisci enim quam ut ab
- molestiae perspiciatis ipsa
- illo est ratione doloreque quia maiores aut

10 séances suivies sur 10

Masquer

Un mot sur l'IA

Et l'IA dans tout cela ?



- Codex, Claude Code, Gemini, Grok : les solutions se multiplient
- Un prompt engendre une application en quelques minutes
- Le risque : produire du code sans le comprendre
- Oui, mais...

Acteur ou spectateur ?

- Perte progressive des connaissances
- Incapacité de comprendre où et comment corriger le code
- Risques de sécurité et de protection des données
- Panne des services IA : chômage technique
- L'entreprise bloque l'accès à l'IA : chômage technique, à nouveau
- En entretien d'embauche : l'illusion tombe

IA : un usage raisonnable

- Un outil précieux, utilisé à bon escient
- Résumer, rechercher, expliquer, tester, relire
- Vous restez aux commandes de vos projets
- Cette formation a été **assistée** par l'IA, pas générée par l'IA
- Durant l'examen, l'IA ne sera pas autorisée

Plan de la formation



Bloc 1 en console, bloc 2 dans la page, bloc 3 avec des données réelles

Approche agile



1. Les bases du Web dynamique



1. Les bases du Web dynamique

Les langages : niveaux, domaines, paradigmes

Qu'est-ce que JavaScript ?

Pourquoi apprendre le JavaScript ?

Installer son environnement

La console

Les expressions

Les variables et leurs types

Les opérateurs, les conditions

Les listes

Les boucles

Les primitives

L'objet Math

Les fonctions, les callbacks

Les portées des variables (scope)

Les objets

2. JavaScript et le DOM



2. JavaScript et le DOM

- JavaScript dans le navigateur
- Sélectionner des éléments
- Changer le contenu et les styles
- Les évènements
- Créer et supprimer des éléments
- Les formulaires et les validations
- Délégation d'évènements et parcours du DOM

3. Applications Web



3. Applications Web

- Le mode asynchrone
- Accès aux API
- Accès aux bases de données
- L'écosystème : React, Vue, Angular

Les labos : trois niveaux

Même notion, trois profondeurs. Tout le monde fait le niveau 1, puis chacun avance à son rythme.

Niveau 1 · Simple

L'essentiel

- Appliquer la notion telle que vue
- Objectif : tout le monde le termine

Niveau 2 · Modéré

Consolider

- Un mini-scénario concret
- Combiner avec les notions précédentes

Niveau 3 · Avancé

Le défi

- Prédire, déboguer, contourner
- Un bonus pour creuser la documentation

Règle : uniquement ce qui a déjà été vu. Les énoncés d'abord, les solutions ensuite.

JavaScript

1. Les bases du Web dynamique

Les langages : niveaux, domaines, paradigmes

Qu'est-ce que JavaScript ?

Pourquoi apprendre le JavaScript ?

Installer son environnement

La console

Les expressions

Les variables et leurs types

Les opérateurs, les conditions

Les listes

Les boucles

Les primitives

L'objet Math

Les fonctions, les callbacks

Les portées des variables (scope)

Les objets

Qu'est-ce que JavaScript ?

- JavaScript, ou JS, est un **langage de programmation**
- Il s'exécute dans les navigateurs, sur des serveurs, dans des applications mobiles et sur des équipements embarqués
- Dans une page, il complète HTML et CSS

HTML

Structure et contenu

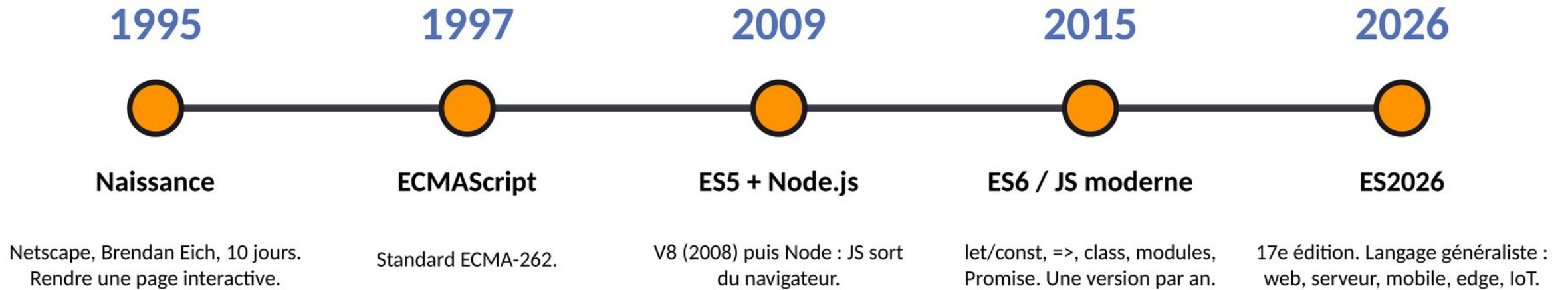
CSS

Mise en forme

JavaScript

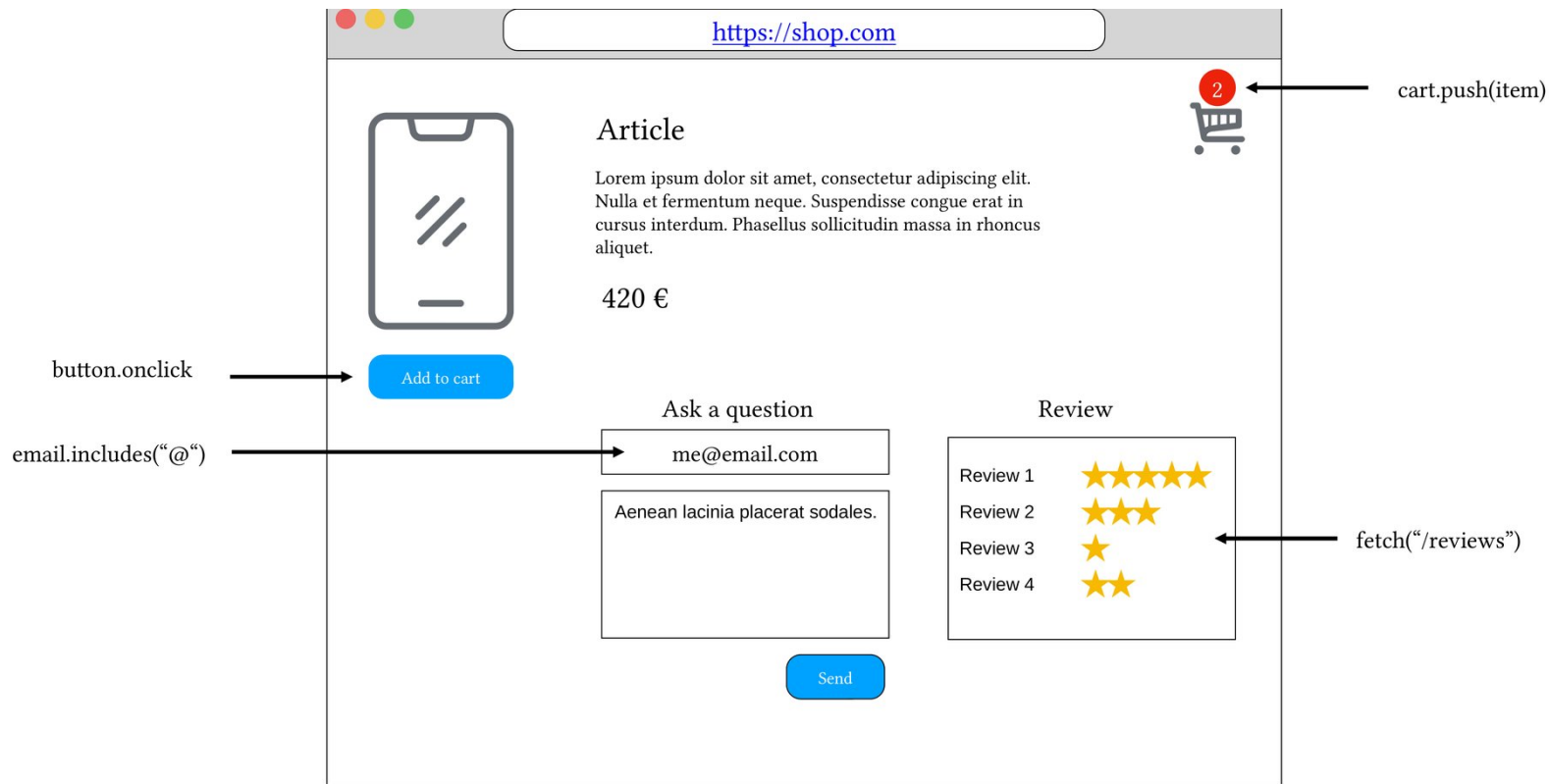
Logique et comportement

Historique de JavaScript



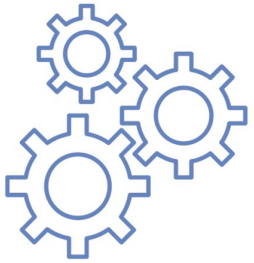
Standard ECMA-262 : une édition par an depuis 2015. La question utile : « cette fonctionnalité est-elle supportée par mes navigateurs cibles ? »

JavaScript dans une page

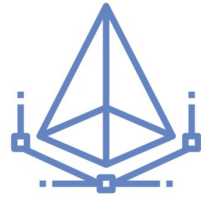


- `button.onclick` : réagir à un clic
- `cart.push(item)` : mettre à jour le panier
- `email.includes("@")` : valider une saisie
- `fetch("/reviews")` : charger des données depuis le serveur

Structure d'un navigateur



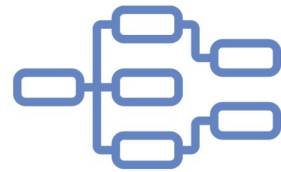
Moteur JavaScript



Moteur de rendu



Web API



Event Loop

- **Moteur de rendu** (Blink, Gecko, WebKit) : HTML et CSS deviennent des pixels
- **Moteur JavaScript** (V8, SpiderMonkey, JavaScriptCore) : exécute le code
- **Web APIs** (DOM, fetch, setTimeout, localStorage) : fournies par le navigateur, pas par le langage
- **Event Loop** : orchestre l'exécution des tâches sur le thread JavaScript principal

Le moteur JavaScript

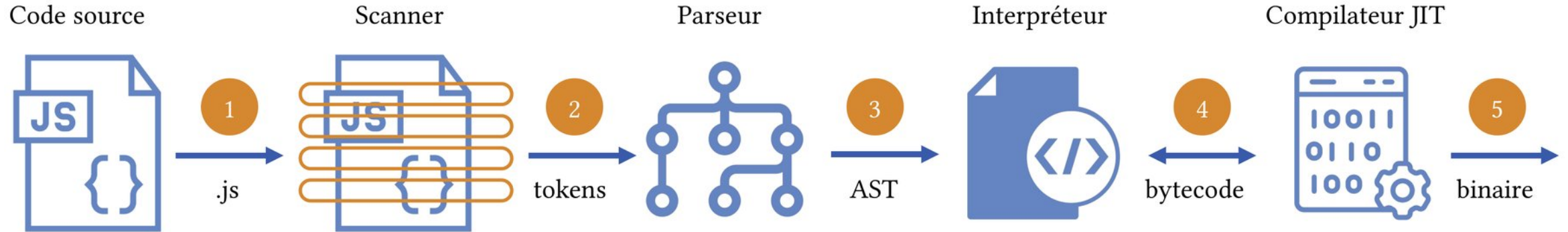
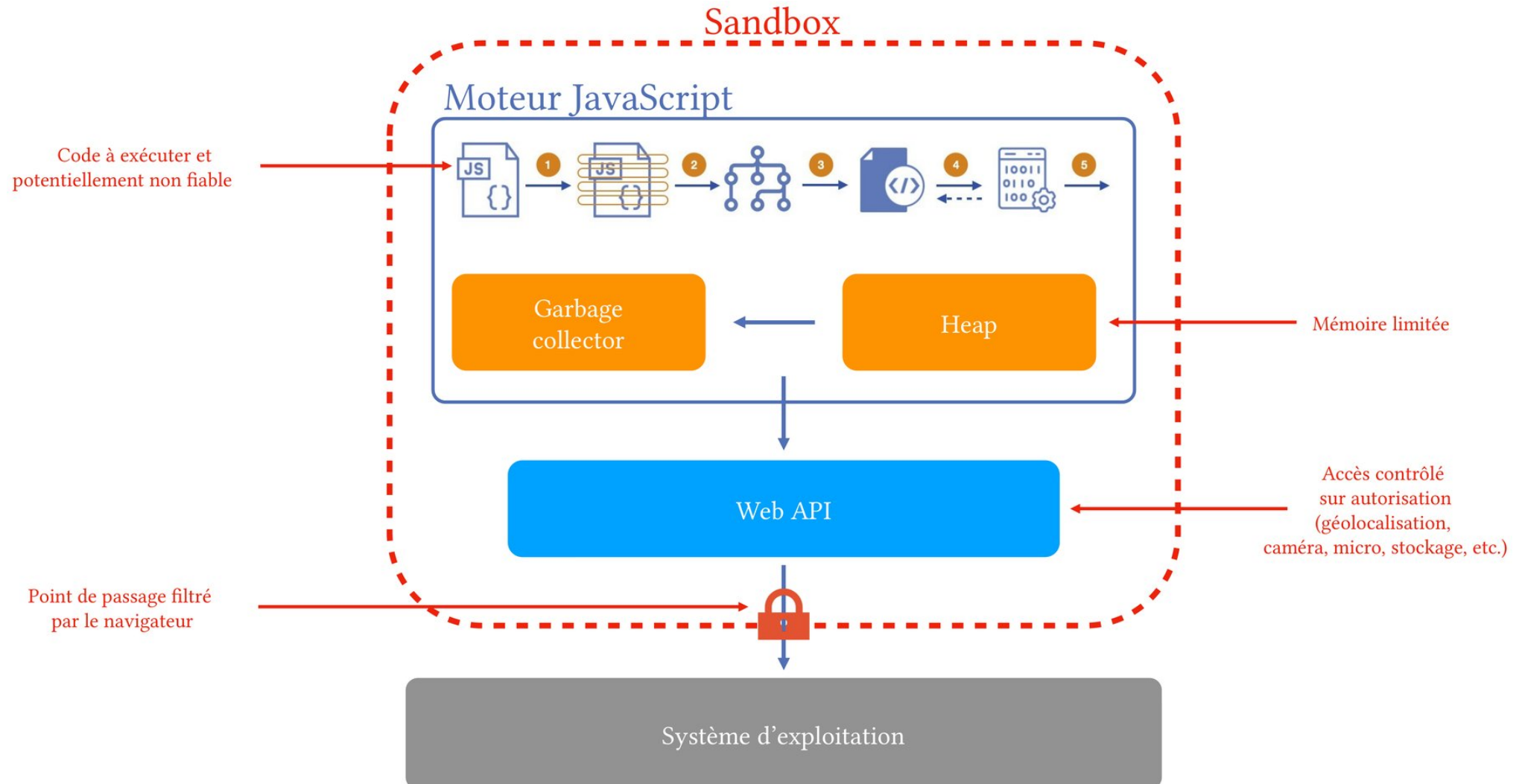


Schéma simplifié d'un moteur JavaScript moderne

Une `SyntaxError` est détectée par le parseur, avant l'exécution : le programme ne démarre pas

Le bac à sable du navigateur

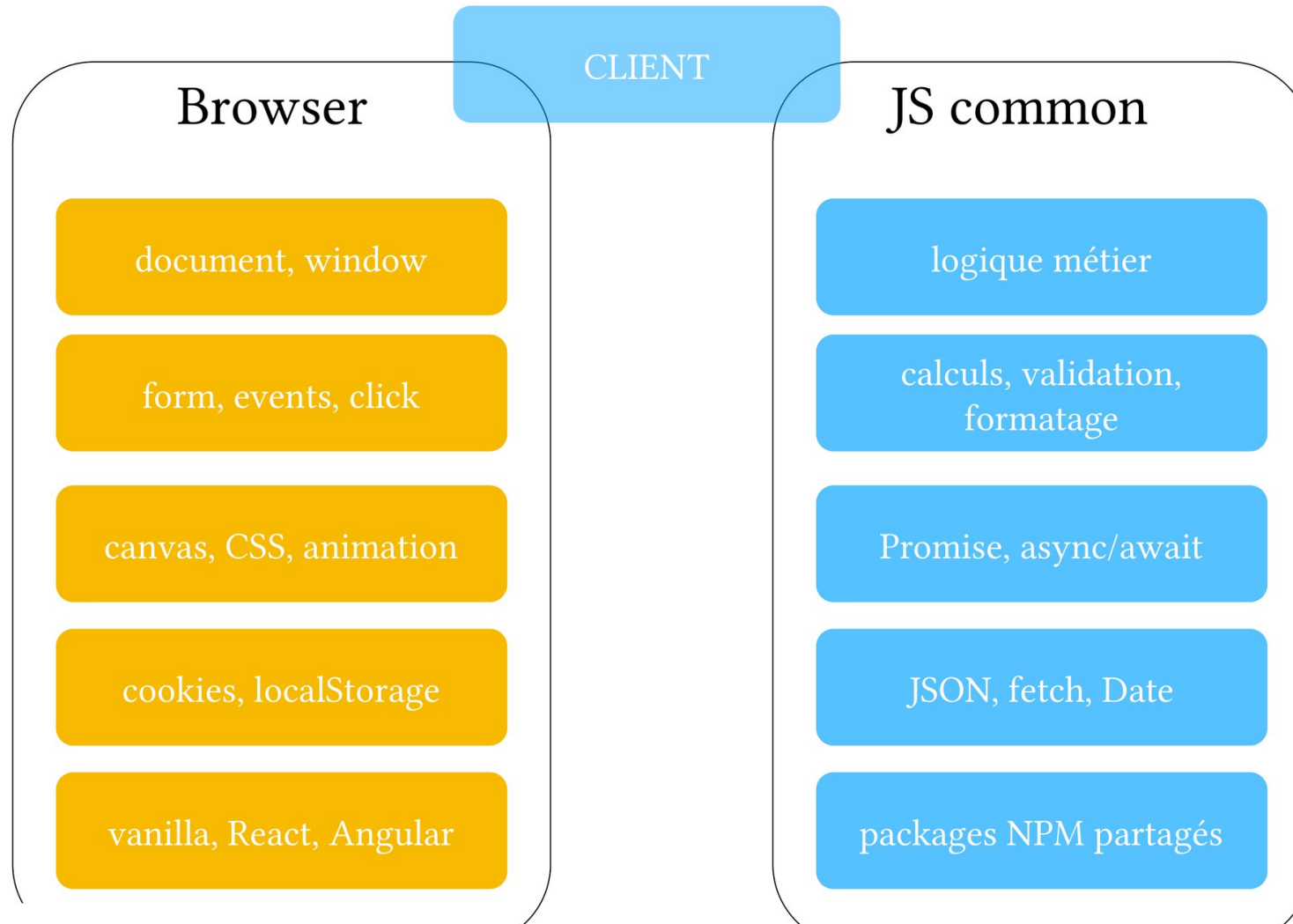


Les différents moteurs

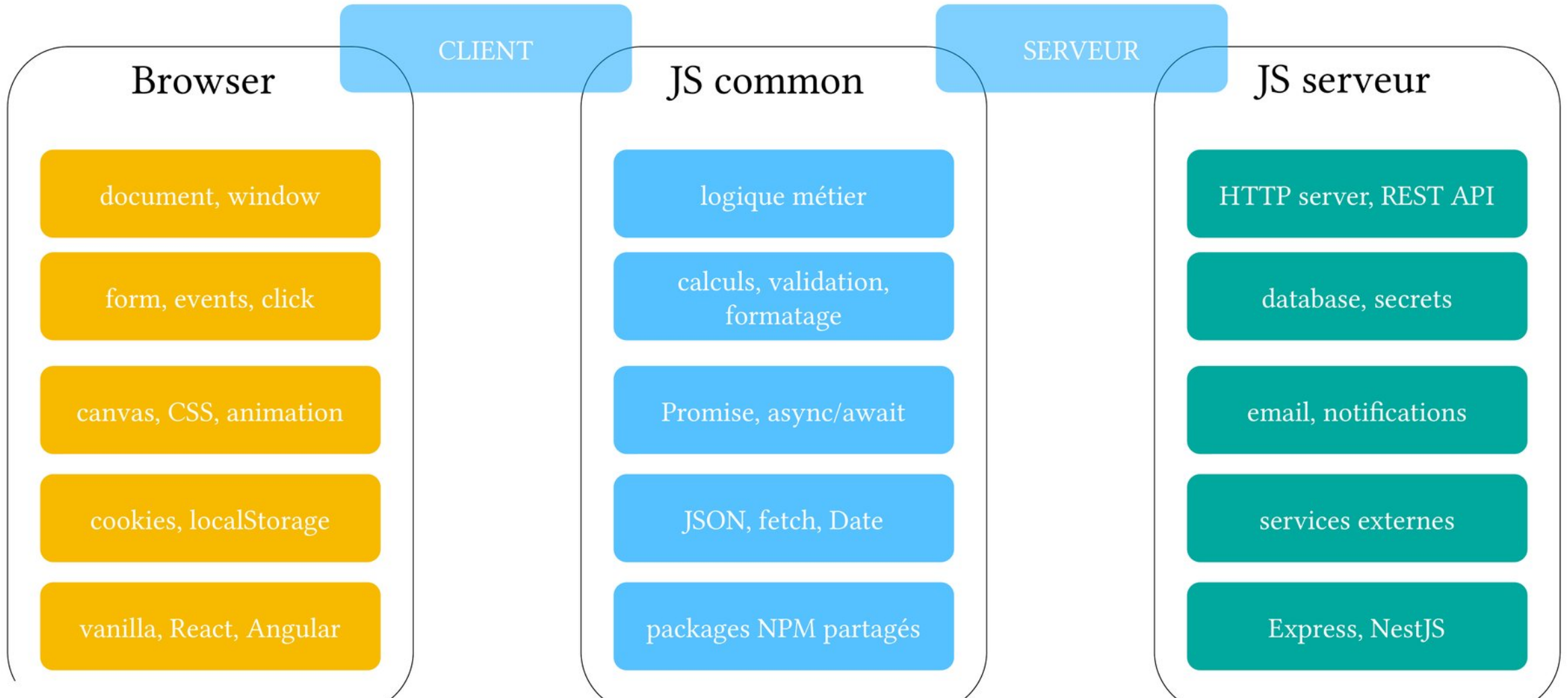
Moteur JS	Moteur de rendu	Éditeur	Navigateurs
V8	Blink (Chromium)	Google	Chrome, Edge, Opera, Brave, Vivaldi, Arc, Samsung Internet
SpiderMonkey	Gecko	Mozilla	Firefox, Tor Browser, Zen
JavaScriptCore (Nitro)	WebKit	Apple	Safari et tous les autres navigateurs sur iOS *

(*) Les autres navigateurs iOS utilisent généralement WebKit ; des moteurs alternatifs sont désormais autorisés dans certaines régions, notamment l'UE.

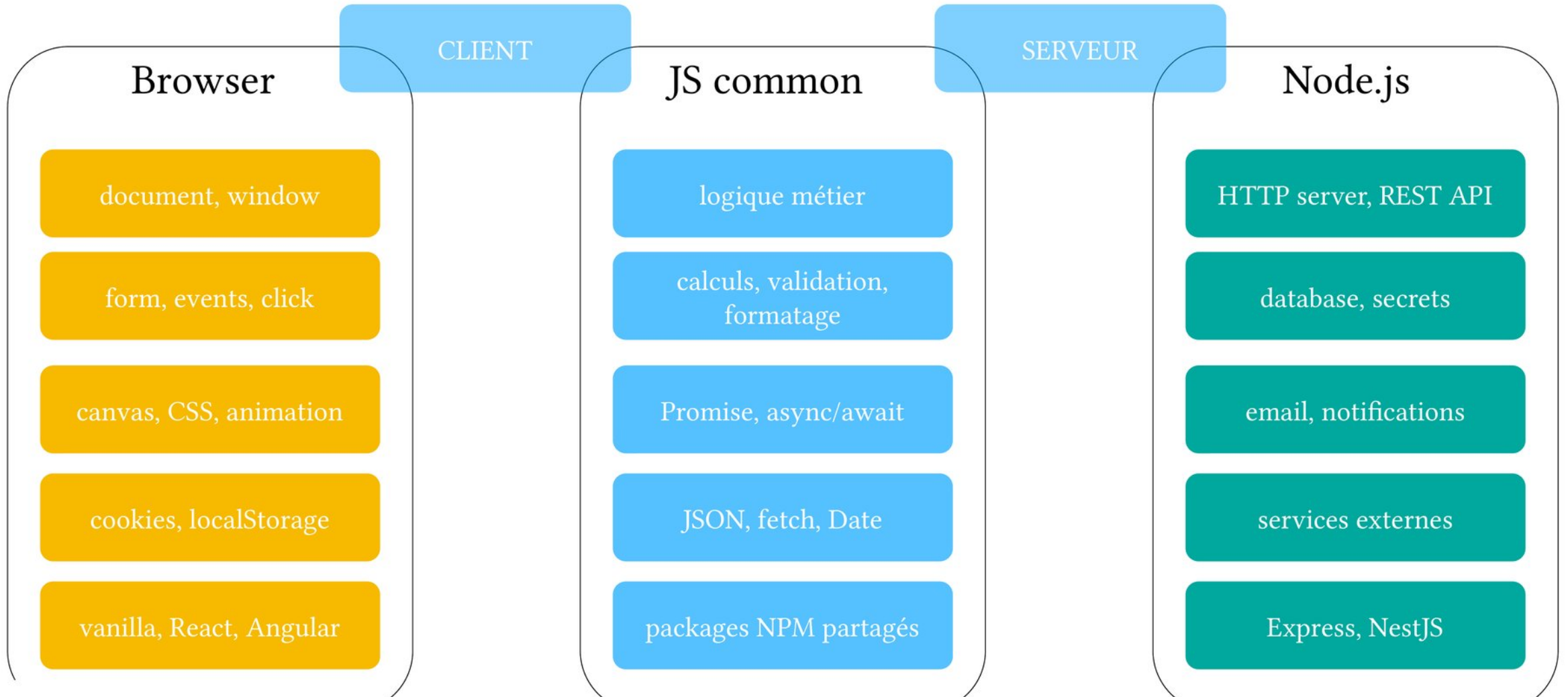
Fonctionnement côté client



Responsabilités côté serveur



Cette formation : Node.js



Node.js

- Une application a besoin de **plus** que le navigateur : données, e-mails, authentification, services distants
- Le navigateur est volontairement **limité** : accès au système, secrets, confiance
- **Node.js** exécute JavaScript hors du navigateur
- Deux usages : sur votre **laptop** pour développer, sur un serveur pour faire tourner l'application

En résumé

HTML

Langage de balisage :
structure et contenu

CSS

Langage de style :
présentation

JavaScript

Langage de
programmation : logique
et comportement

Client / serveur

Le navigateur demande,
le serveur répond :
statique ou dynamique

Moteur JavaScript

Exécute JavaScript dans
le navigateur, dans un bac
à sable

Node.js

Exécute JavaScript hors
du navigateur :
développement et serveur

Installer son environnement

Labo 1 : l'environnement de travail

Niveau 1 · Simple

La console du navigateur

- Ouvrez la console de votre navigateur (Chrome, Firefox ou Safari)
- Entrez `about:blank` comme URL
- Tapez `console.log("Hello")` puis Enter
- Vérifiez que le message s'affiche

Niveau 2 · Modéré

Node.js et npm

- Téléchargez la version LTS sur nodejs.org
- Installez avec les options par défaut
- Ouvrez un terminal : PowerShell ou Terminal
- Vérifiez avec `node --version` puis `npm --version`

Niveau 3 · Avancé

VS Code et ses extensions

- Téléchargez Visual Studio Code sur code.visualstudio.com
- Installez les extensions Code Runner, ESLint, Prettier et Live Server
- Créez un dossier de travail pour les labos du cours
- Ouvrez ce dossier dans VS Code

Labo 1 · niveau 1

Niveau 1 · Simple

La console du navigateur

- Ouvrez la console de votre navigateur (Chrome, Firefox ou Safari)
- Entrez `about:blank` comme URL
- Tapez `console.log("Hello")` puis Enter
- Vérifiez que le message s'affiche

Hint

Chrome : `Ctrl + Shift + J`
(macOS : `⌘ + ⌘ + J`). Firefox :
`Ctrl + Shift + K` (macOS : `⌘ + ⌘ + K`). Safari : activer d'abord le menu Développement.

Labo 1 · niveau 2

Niveau 2 · Modéré

Node.js et npm

- Téléchargez la version LTS sur nodejs.org
- Installez avec les options par défaut
- Ouvrez un terminal : PowerShell ou Terminal
- Vérifiez avec `node --version` puis `npm --version`

Hint

LTS = Long Term Support, la version stable. npm est installé en même temps que Node.js.

Labo 1 · niveau 3

Niveau 3 · Avancé

VS Code et ses extensions

- Téléchargez Visual Studio Code sur code.visualstudio.com
- Installez les extensions Code Runner, ESLint, Prettier et Live Server
- Créez un dossier de travail pour les labos du cours
- Ouvrez ce dossier dans VS Code

Hint

Les extensions s'installent depuis l'icône Extensions de la barre latérale. Code Runner a besoin de Node.js : c'est pour cela qu'il vient après.

Solution labo 1 · niveau 1

Niveau 1 · Simple

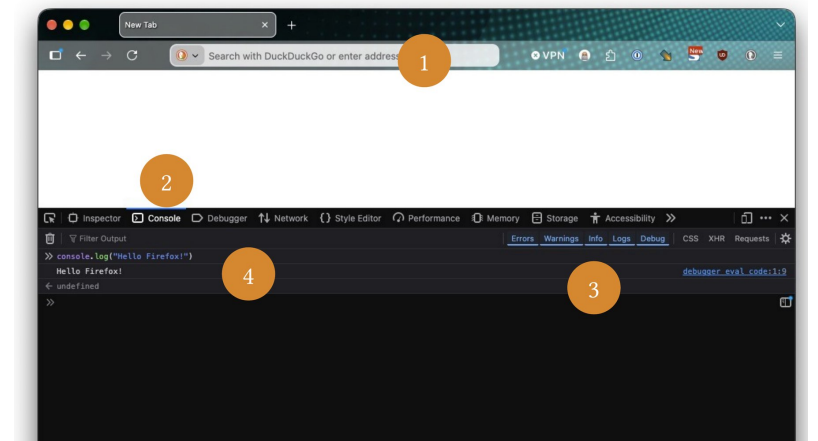
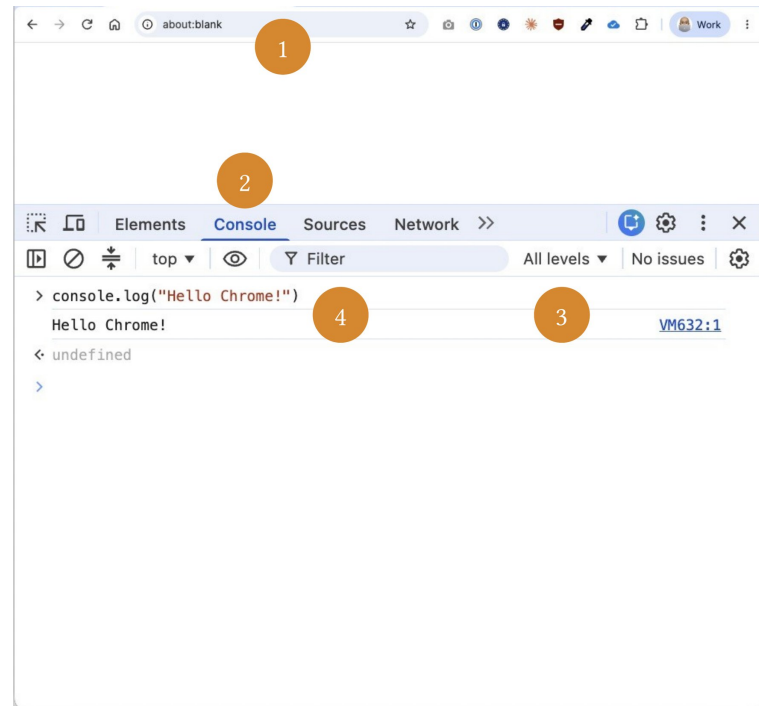
1. Entrer **about:blank** comme URL
2. Sélectionner l'onglet Console
3. Le niveau Info doit être sélectionné
4. Entrer le code JavaScript puis Enter



Chrome : **Ctrl + Shift + J**
macOS : **⌘ + ⌘ + J**



Firefox : **Ctrl + Shift + K**
macOS : **⌘ + ⌘ + K**



Solution labo 1 - niveau 1 (Safari)

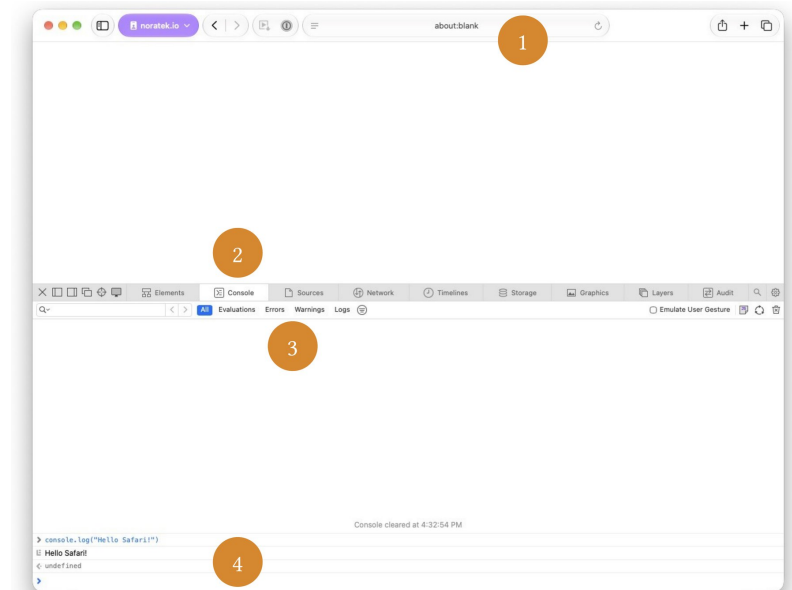
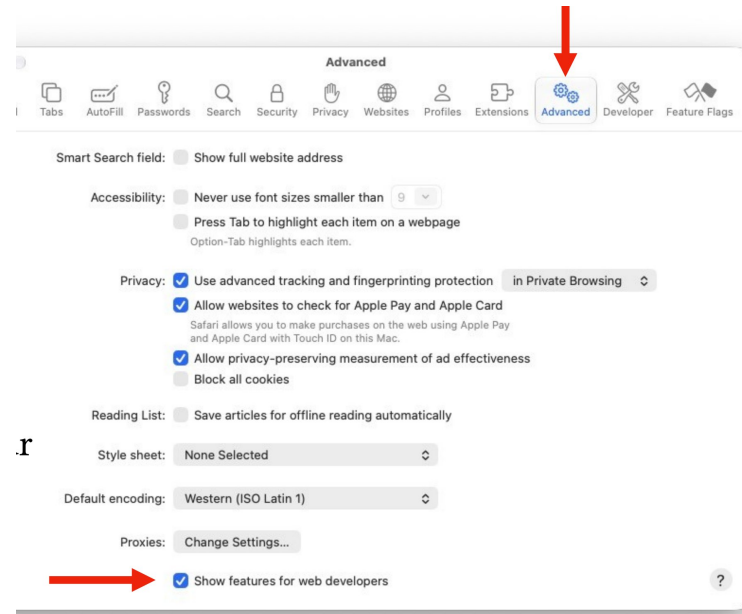
Niveau 1 - Simple



Safari : ⌘ + ⌘ + C

Réglages > Avancé : cocher
« Afficher les fonctionnalités pour les développeurs »

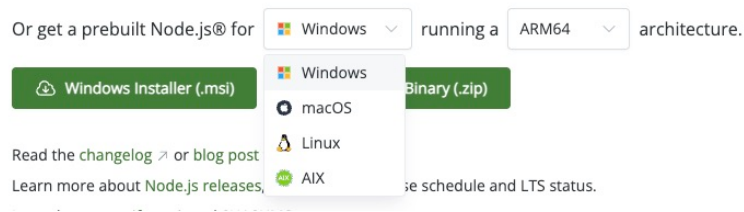
Le niveau **All** doit être sélectionné



Solution labo 1 · niveau 2

Niveau 2 · Modéré

Dans un terminal (PowerShell sous Windows, Terminal sous macOS et Linux) :



nodejs.org : choisir la version **LTS**, installer avec les options par défaut. npm est installé en même temps.

```
> node --version  
v24.16.0
```

```
> npm --version  
11.13.0
```

Windows : npm est bloqué

Niveau 2 · Modéré

Si cette erreur apparaît :

```
running scripts is disabled on this system
```

Exécuter dans PowerShell :

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Puis réessayer :

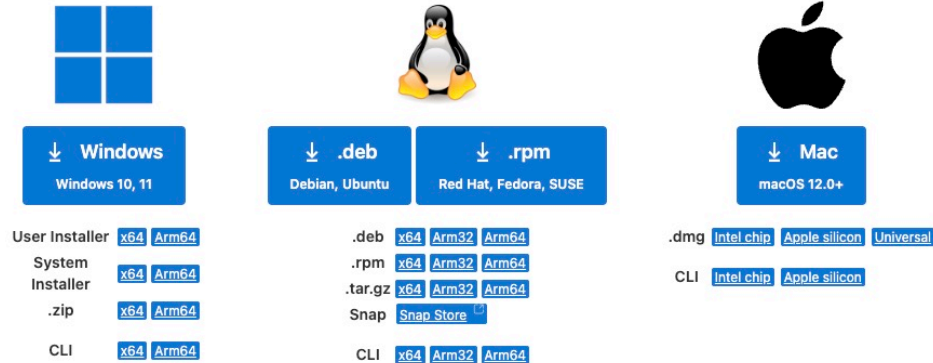
```
npm --version  
11.13.0
```

Solution labo 1 · niveau 3

Niveau 3 · Avancé

Download Visual Studio Code

Free and built on open source. AI agents, Git, debugging, and extensions included.



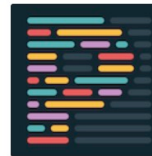
code.visualstudio.com : télécharger la version correspondant à votre système, puis créer un dossier de travail, par exemple [Projects/ifapme-x64-pwf](#).



Code Runner (Jun Han) : exécuter rapidement du JavaScript depuis VS Code. Nécessite Node.js.



ESLint (Microsoft) : détecter les erreurs et signaler les mauvaises pratiques.



Prettier - Code formatter : formater automatiquement le code.



Live Server (Ritwick Dey) : lancer une page web locale avec rechargement automatique.

L'environnement est prêt

- Node.js est installé
- npm est disponible
- Visual Studio Code est prêt
- Nous pouvons écrire et exécuter du JavaScript
- Première étape : interagir avec la console

La console et console.log

- La console affiche les informations produites par le code
- `console.log()` affiche une ou plusieurs valeurs
- Même fonctionnement dans le navigateur et avec Node.js

```
console.log("Hello, World!")  
console.log(42)  
console.log(true)
```

```
Hello, World!  
42  
true
```

SORTIE

Le point-virgule

- Le `;` est généralement optionnel en JavaScript moderne
- Certains projets l'utilisent encore par convention
- Dans ce cours : sans point-virgule

```
console.log("Hello, World!")  
console.log(42)  
console.log(true)
```

=

```
console.log("Hello, World!");  
console.log(42);  
console.log(true);
```

log, info, warn et error

- **console** est un objet fourni par l'environnement JavaScript
- Il propose plusieurs méthodes pour afficher des informations
- **log** : message général
- **info** : message d'information
- **warn** : avertissement
- **error** : erreur

```
console.log("Chargement de la page")  
console.info("3 nouveaux messages")  
console.warn("Le formulaire est incomplet")  
console.error("Le serveur ne répond pas")
```

```
Chargement de la page  
3 nouveaux messages  
Le formulaire est incomplet  
Le serveur ne répond pas
```

SORTIE

Les expressions

- Une expression **produit une valeur**
- JavaScript évalue l'expression, puis produit le résultat
- `console.log` affiche ce résultat

```
console.log(5)  
console.log(2 + 3)  
console.log("Hello " + "world")
```

```
5  
5  
Hello world
```

SORTIE

Calculs et concaténation

- Calculs : **+** **-** ***** **/**
- Les règles de priorité sont celles des mathématiques
- Les parenthèses contrôlent l'ordre
- Avec du texte, **+** permet de **concaténer**

```
console.log(2 + 3 * 4)           // 14
console.log((2 + 3) * 4)        // 20
console.log(10 / 4)             // 2.5

console.log("Bonjour " + "Sara")
// Bonjour Sara
console.log("Total : " + 12 + " €")
// Total : 12 €
```

Labo 2 : expressions et console

Niveau 1 · Simple

Premiers affichages

- Créez un fichier `labo-2-1.js`
- Affichez avec `console.log` : une concaténation de textes, une addition, une division, une expression avec parenthèses
- Exécutez le fichier avec Code Runner, puis avec Node.js

Niveau 2 · Modéré

Le devis

- Un devis de refonte : 3 jours à 400 € par jour
- Affichez le montant hors TVA, la TVA (21 %) et le total TTC, chacun sur une ligne avec un libellé
- Terminez par une information (`console.info`) et un avertissement (`console.warn`)

Niveau 3 · Avancé

Prédire avant d'exécuter

- Écrivez d'abord votre prédiction en commentaire, puis vérifiez :
- `2 + 3 * 4` et `(2 + 3) * 4`
- `"Total : " + 2 + 3` et `"Total : " + (2 + 3)`
- `2 + 3 + " €"`
- Bonus : que donne `0.1 + 0.2` ?

Labo 2 · niveau 1

Niveau 1 · Simple

Premiers affichages

- Créez un fichier `labo-2-1.js`
- Affichez avec `console.log` : une concaténation de textes, une addition, une division, une expression avec parenthèses
- Exécutez le fichier avec Code Runner, puis avec Node.js

Hint

Une instruction par ligne. Code Runner : icône Run en haut à droite. Node.js : `node labo-2-1.js` dans le terminal.

Labo 2 · niveau 2

Niveau 2 · Modéré

Le devis

- Un devis de refonte : 3 jours à 400 € par jour
- Affichez le montant hors TVA, la TVA (21 %) et le total TTC, chacun sur une ligne avec un libellé
- Terminez par une information (`console.info`) et un avertissement (`console.warn`)

Hint

Un libellé est un texte : `"TVA = "`
`+ 3 * 400 * 0.21 + " €"`.
Le calcul s'exécute avant la concaténation.

Labo 2 · niveau 3

Niveau 3 · Avancé

Prédire avant d'exécuter

- Écrivez d'abord votre prédiction en commentaire, puis vérifiez :
- `2 + 3 * 4` et `(2 + 3) * 4`
- `"Total : " + 2 + 3` et `"Total : " + (2 + 3)`
- `2 + 3 + " €"`
- Bonus : que donne `0.1 + 0.2` ?

Hint

JavaScript lit de gauche à droite.
Dès qu'un texte est rencontré, `+`
colle au lieu d'additionner.

Solution labo 2 · niveau 1

Niveau 1 · Simple

```
console.log("Bonjour " + "IFAPME")  
console.log(40 + 2)  
console.log(10 / 4)  
console.log((2 + 3) * 4)
```

```
Bonjour IFAPME  
42  
2.5  
20
```

SORTIE

Solution labo 2 · niveau 2

Niveau 2 · Modéré

```
console.log("Devis : refonte du site")
console.log("3 jours x 400 € = " + 3 * 400 + " €")
console.log("TVA 21 % = " + 3 * 400 * 0.21 + " €")
console.log("Total TTC = " + 3 * 400 * 1.21 + " €")
console.info("Devis valable 30 jours")
console.warn("Acompte de 30 % à la commande")
```

```
Devis : refonte du site
3 jours x 400 € = 1200 €
TVA 21 % = 252 €
Total TTC = 1452 €
Devis valable 30 jours
Acompte de 30 % à la commande
```

SORTIE

Solution labo 2 · niveau 3

Niveau 3 · Avancé

```
console.log(2 + 3 * 4)           // 14
console.log((2 + 3) * 4)        // 20
console.log("Total : " + 2 + 3) // Total : 23
console.log("Total : " + (2 + 3)) // Total : 5
console.log(2 + 3 + " €")       // 5 €
console.log(0.1 + 0.2)          // 0.30000000000000004
```

```
14
20
Total : 23
Total : 5
5 €
0.30000000000000004
```

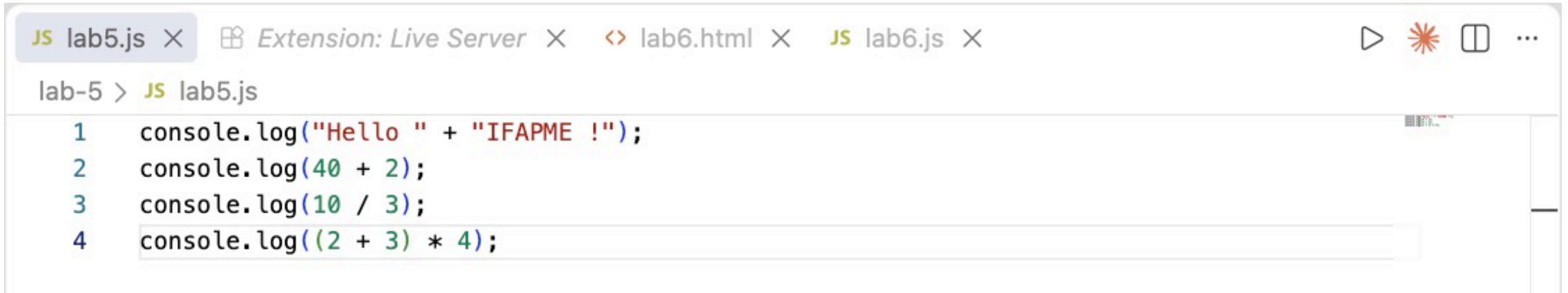
SORTIE

À retenir

- `"Total : " + 2 + 3` : dès que le texte est rencontré, `+` concatène, le `3` est collé
- `0.1 + 0.2` : les nombres décimaux sont approximés en binaire ; on y reviendra avec l'objet `Math`

Exécuter avec Code Runner

Ouvrir le fichier et cliquer sur l'icône **Run**, en haut à droite de l'éditeur :



```
JS lab5.js X  Extension: Live Server X  <> lab6.html X  JS lab6.js X
lab-5 > JS lab5.js
1 console.log("Hello " + "IFAPME !");
2 console.log(40 + 2);
3 console.log(10 / 3);
4 console.log((2 + 3) * 4);
```

La fenêtre **Output** affiche le résultat. Les lignes Running et Done sont ajoutées par Code Runner :

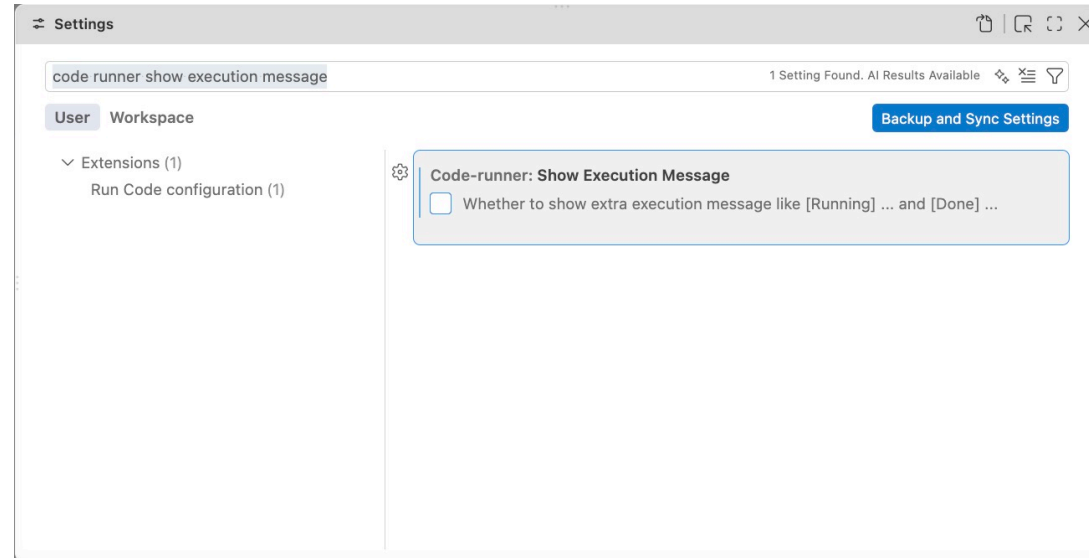


```
Problems Output ...  Filter (e.g. text, !excludeTex...  Code
[Running] node "/Users/eloudsa/Projects/ifapme-x64-pwf/lab-5/lab5.js"
Hello IFAPME !
42
3.3333333333333335
20

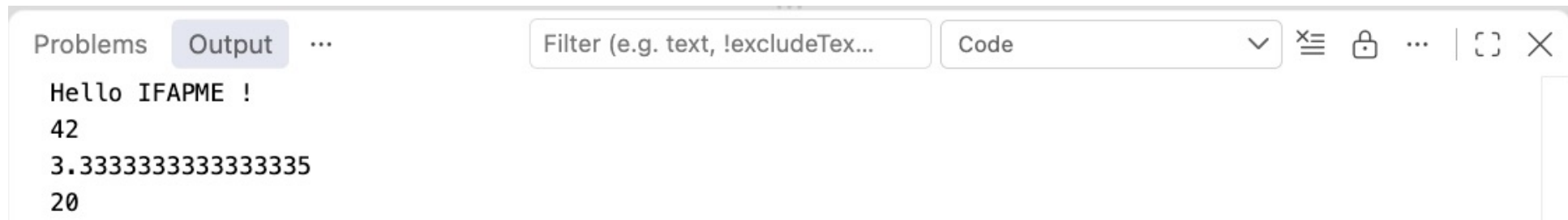
[Done] exited with code=0 in 0.052 seconds
```

Code Runner : masquer Running et Done

Dans les réglages de VS Code, rechercher `code runner show execution message` et décocher la case :



Exécuter à nouveau : seule la sortie du programme reste.



Exécuter avec Node.js

Ouvrir le terminal de VS Code (menu Terminal > New Terminal) et vérifier que le dossier courant est celui du fichier :



```
Problems Output Debug Console Terminal Ports zsh - lab-5 + - [ ] [ ] ... [ ] [ ] X
~/Projects/ifapme-x64-pwf 18:12
> pwd
/Users/eloudsa/Projects/ifapme-x64-pwf
~/Projects/ifapme-x64-pwf 18:12
> cd lab-5
~/Projects/ifapme-x64-pwf/lab-5 v24.16.0 18:12
> ls
lab5.html    lab5.js
~/Projects/ifapme-x64-pwf/lab-5 v24.16.0 18:12
>
```

Exécuter le fichier avec **node**, avec ou sans son extension :



```
Problems Output Debug Console Terminal Ports zsh - lab-5 + - [ ] [ ] ... [ ] [ ] X
~/Projects/ifapme-x64-pwf/lab-5 v24.16.0 18:13
> node lab5.js
Hello IFAPME !
42
3.3333333333333335
20
~/Projects/ifapme-x64-pwf/lab-5 v24.16.0 18:13
> node lab5
Hello IFAPME !
42
3.3333333333333335
20
```

Les commentaires

- `//` a déjà été aperçu dans les exemples précédents
- Un commentaire est un texte destiné aux humains, **ignoré** par JavaScript
- `//` : jusqu'à la fin de la ligne
- `/* ... */` : sur une ou plusieurs lignes

```
// Un commentaire sur une ligne
console.log("Bonjour")    // en fin de ligne

/*
  Un commentaire
  sur plusieurs lignes
*/
console.log("Au revoir")
```

```
Bonjour
Au revoir
```

SORTIE

Pourquoi commenter ?

- Expliquer une **intention** ou une **décision**
- Laisser une note : `// TODO: ...`
- Désactiver temporairement une ligne de code
- Éviter les commentaires inutiles si le code est déjà clair

```
// Tarif réduit appliqué aux apprenants
console.log("Prix : " + 40 * 0.5 + " €")

// TODO: confirmer le taux de TVA

// console.log("ligne désactivée")
```

```
Prix : 20 €
```

SORTIE

Labo 3 : les commentaires

Niveau 1 · Simple

Annoter le devis

- Repartez du devis du labo 2 (niveau 2) dans `labo-3-1.js`
- Ajoutez un commentaire sur une ligne en tête de fichier
- Ajoutez un commentaire en fin d'instruction
- Exécutez : la sortie ne doit pas changer

Niveau 2 · Modéré

Désactiver sans supprimer

- Désactivez l'avertissement avec `//`
- Désactivez les lignes TVA et Total TTC avec un seul commentaire `/* ... */`
- Exécutez et comparez la sortie
- Réactivez le tout : la sortie initiale revient

Niveau 3 · Avancé

Le bloc de documentation

- Ajoutez en tête de fichier un bloc `/* ... */` : objet du fichier, auteur, date
- Ajoutez une note `// TODO:` sur le taux de TVA
- Prédiction : que fait `console.log("Voir https://nodejs.org")` ? Vérifiez
- Bonus : que se passe-t-il avec un bloc `/* ... */` à l'intérieur d'un autre ?

Labo 3 · niveau 1

Niveau 1 · Simple

Annoter le devis

- Repartez du devis du labo 2 (niveau 2) dans `labo-3-1.js`
- Ajoutez un commentaire sur une ligne en tête de fichier
- Ajoutez un commentaire en fin d'instruction
- Exécutez : la sortie ne doit pas changer

Hint

`//` s'arrête à la fin de la ligne : tout ce qui suit, sur la même ligne, est ignoré.

Labo 3 · niveau 2

Niveau 2 · Modéré

Désactiver sans supprimer

- Désactivez l'avertissement avec `//`
- Désactivez les lignes TVA et Total TTC avec un seul commentaire `/* ... */`
- Exécutez et comparez la sortie
- Réactivez le tout : la sortie initiale revient

Hint

Un bloc `/* ... */` peut couvrir plusieurs instructions d'un coup.

Labo 3 · niveau 3

Niveau 3 · Avancé

Le bloc de documentation

- Ajoutez en tête de fichier un bloc `/* ... */` : objet du fichier, auteur, date
- Ajoutez une note `// TODO:` sur le taux de TVA
- Prédiction : que fait `console.log("Voir https://nodejs.org")` ? Vérifiez
- Bonus : que se passe-t-il avec un bloc `/* ... */` à l'intérieur d'un autre ?

Hint

JavaScript lit les guillemets avant les `//`. Un `//` à l'intérieur d'une chaîne fait partie du texte.

Solution labo 3 · niveau 1

Niveau 1 · Simple

```
// Devis de refonte : 3 jours à 400 € par jour
console.log("Devis : refonte du site")
console.log("3 jours x 400 € = " + 3 * 400 + " €") // hors TVA
console.log("TVA 21 % = " + 3 * 400 * 0.21 + " €")
console.log("Total TTC = " + 3 * 400 * 1.21 + " €")
console.info("Devis valable 30 jours")
console.warn("Acompte de 30 % à la commande")
```

```
Devis : refonte du site
3 jours x 400 € = 1200 €
TVA 21 % = 252 €
Total TTC = 1452 €
Devis valable 30 jours
Acompte de 30 % à la commande
```

SORTIE

Solution labo 3 · niveau 2

Niveau 2 · Modéré

```
// Devis de refonte : 3 jours à 400 € par jour
console.log("Devis : refonte du site")
console.log("3 jours x 400 € = " + 3 * 400 + " €") // hors TVA
/*
console.log("TVA 21 % = " + 3 * 400 * 0.21 + " €")
console.log("Total TTC = " + 3 * 400 * 1.21 + " €")
*/
console.info("Devis valable 30 jours")
// console.warn("Acompte de 30 % à la commande")
```

```
Devis : refonte du site
3 jours x 400 € = 1200 €
Devis valable 30 jours
```

SORTIE

Solution labo 3 · niveau 3

Niveau 3 · Avancé

```
/*
  Devis de refonte du site
  Auteur : Sara
  Date : septembre 2026
*/

// TODO: confirmer le taux de TVA avec le client
console.log("Devis : refonte du site")
console.log("Total TTC = " + 3 * 400 * 1.21 + " €")
console.log("Voir https://nodejs.org") // le // est du texte

// Bonus : un bloc dans un bloc
// /* a /* b */ c */ → SyntaxError
```

```
Devis : refonte du site
Total TTC = 1452 €
Voir https://nodejs.org
```

SORTIE

À retenir

- Dans une chaîne, `//` n'est pas un commentaire : l'URL s'affiche en entier
- Les blocs `/* ... */` ne s'imbriquent pas : le premier `*/` ferme le bloc, le reste provoque une `SyntaxError`

En résumé

Client et serveur

Le navigateur demande, le serveur répond : HTML et CSS statiques, JavaScript pour le dynamique

Environnement

Node.js, npm et VS Code installés et vérifiés ; Code Runner pour exécuter

La console

`console.log` affiche le résultat du code ; `info`, `warn`, `error` pour nuancer

Les expressions

Valeurs et opérateurs produisent un résultat ; `+` additionne ou concatène

Les commentaires

`//` et `/* ... */` : expliquer, noter, désactiver, sans être exécutés

Prochaine session

Les variables et leurs types : nommer une valeur pour la réutiliser